

基于机器学习算法和统计模型的文物归类还原模型

摘要

本文针对已风化玻璃文物化学成分参数的还原和两种玻璃的分类问题，建立基于成分含量的分类模型和预估模型，之后基于误差最小、最符合实际的原则建立高钾和铅钡玻璃的高维聚类分类模型并进行求解，最后给出文物的最优分化策略，并得出每个类别的化学成分之间的精准的关联关系。建立和求解模型的具体思路如下：

针对问题一的文物属性与风化程度的关联性问题，对纹饰、颜色、玻璃类型与是否风化的关系进行简化，利用非参数检验方法建立了基于卡方检验的关联性检验模型。然后根据化学成分参数的统计规律构建预测模型。求解则使用卡方检验算法，计算出文物每个属性与风化程度的卡方统计量，并用岭回归预测求解出风化前文物各化学成分的组成情况。

针对问题二的亚类分类问题，分别对高钾组和铅钡组建立了高维聚类分析模型。聚类模型的求解，本文使用随机森林——Kmeans 算法，计算出效果最好的亚类分类数值，并通过主成分分析将高维成分特征降维得到所有合理样本的分类散点图。

针对问题三将未知类别文物分类的问题，把样本分为两种模型相互比较验证，选择对变量空间进行降维并回归的方法进行建模。模型主要用到主成分回归（PCR）与偏最小二乘回归（PLS）的方法建立预测模型并进行对比分析，得到完整分类。

针对问题四，利用正态性分布，求得化学成分之间的统计关系，即度量化学成分变量之间的相关性。先对数据进行正态性检验，使用 Spearman 秩相关系数、相关性分析方法得出热力图，展示化学成分关联度结果。

数学模型上，本题主要采用了主成分分析（PCA）、主成分回归（PCR）、偏最小二乘回归（PLS）等建模分析方法。

算法上，本模型主要采用机器学习算法，从化学成分的含量数据中自动分析获得规律，并利用这种分类规律对未知类别的样本数据进行预测。本模型中主要用到了聚类分析、随机森林、有其分析得到的结果可靠性高。除机器学习类算法外，还运用了异常检测算法如 LOF 算法。

此外，此模型具有良好的推广性与广泛适用性，除了应用于玻璃文物，还可以应用于其他种类文物的发掘与保护，除风化反应外，其他化学过程也与本模型有相似之处。

此模型具有准确度高，灵敏性高、合理性强的特点，并创新性地运用数据预处理对高维聚类模型进行了改进。

关键词 卡方检验 改进的聚类模型 随机森林 PLS模型 PCR模型 spearman相关系数

一、问题重述

1.1 问题背景

玻璃是丝绸之路早期贸易往来的宝贵物证，我国留存下来许多宝贵的玻璃文物。玻璃文物主要分为高钾玻璃和铅钡玻璃，其化学成分差异显著，可对文物风化过程中的化学反应造成不同影响；文物纹饰不同，则表面物理结构不同，对风蚀、积水等过程也有一定影响；文物表面颜色对其受光吸热有一定影响。在风化过程中，文物的内部元素与环境元素进行大量交换，导致其成分比例发生变化，从而影响对其类别的正确判断。因此预测风化前文物化学成分含量有利于还原文物原本类别。分析文物的分类规律，又有利于还原未知类别文物的特性，而对考古学家鉴别文物种类有重大意义。

1.2 需要解决的问题

1.2.1 问题一要求分析玻璃文物的风化程度与自身多方面属性之间的关系，且需要建立化学成分预测模型，推测风化前各化学成分的值。

1.2.2 附件给出了所有玻璃文物样本的化学成分及其含量，需要分析出高钾玻璃、铅钡玻璃的分类规律；并分别从中选择最合适、最重要的化学成分，对其进行亚类划分。

1.2.3 问题三要求对未知类别玻璃文物的化学成分进行分析，鉴别其是属于高钾玻璃还是铅钡玻璃，并对分类结果的敏感性进行分析。

1.2.4 针对不同类别的玻璃文物样品，分析其化学成分之间的关联关系，并比较不同类别之间的化学成分关联关系的差异性。

二、问题分析

2.1 问题一的分析

问题一要求分析玻璃文物的风化程度与自身多方面属性之间的关系并预测化学成分含量。拟建立基于卡方检验的关联性检验模型和基于岭回归的成分预测模型。

为了建立完善科学的量化分析体系，该题的思路分为以下三个步骤：

1.数据的优化处理。题目所给数据集中数据有数据部分缺失、数据少量不准确的情况，有以下解决方法：

(1) 对于数据与实际不符的情况，直接将化学成分累加和超出合理范围的数据删除。

(2) 针对化学成分的缺失数据，使用其他数据进行替代将影响到公平性，故在计化学成分含量时计为 0。

2.选择合适的分析算法。为准确评估文物的颜色、纹饰、玻璃类别及化学组成对风化程度的影响，均要对其与风化程度间的关系进行关联性检验。通常用于变量之间差异性分析的模型与方法有，方差分析、T 检验、卡方检验等，由于文物自身属性与分化程度都属于定类变量而非连续变量，因此选择最适合的非参数检验方法中的卡方检验。

3.利用岭回归拟合出成分预测模型。玻璃文物风化后各化学成分含量将发生改变，为准确评估量化出不同分化程度对化学成分的影响方程。

2.2 问题二的分析

问题二要求根据化学成分对文物进行亚类分类。由问题一可知，文物的风化程度主要受类别影响，而决定玻璃文物类别的为其化学成分占比。因此想要在两个大类下进一步细化玻璃文物的亚类，需分析一级分类的分类规律，并根据化学成分细分为数量合适的亚类。拟建立基于随机森林算法的聚类分析模型，通过对所有化学成分的重要度检验，以轮廓系数为主要依据选择分类数量。考虑到样本数量的有限性，分类数量不宜过高。为了获得直观图示，最后再利用 PCA 降维展示亚类分类结果。

2.3 问题三的分析

问题三要求根据文物的化学成分占比，倒推出其所属玻璃文物的类别，与问题二有异曲同工之妙。问题二中已经分好了高钾玻璃和铅钡玻璃两个类别，通过对化学成分的分析找出最优亚类分类方法。问题三是未知分类，要求通过对化学成分的分析找出其原本的一级分类，即是属于高钾玻璃还是铅钡玻璃。针对问题三，步骤如下：

首先将未知类别的样本分为两种模型相互比较验证。由于高钾和铅钡风化前后差距较大，所以将数据分为风化和无风化分别进行建模。

其次，考虑到数据样本少，且特征维度大，选择 PCR 主成分回归模型和 PLS 模型，运用交叉验证策略，分别对风化样本组、未风化样本组分析处理。

最后得出预测结果，并进行误差分析。

2.4 问题四的分析

问题四要求针对高钾、铅钡类别的玻璃文物样品，分别分析每个类别的化学成分之间的关联关系。需要求得化学成分之间的统计关系，即度量化学成分变量之间的相关性，并用适当的统计指标进行展示。本题有两个可能的思路：

1. 在选择相关性分析方法之前，对数据进行正态性检验。若数据近似满足正态性分布，则选择 Pearson 相关系数

2. 若数据很难满足正态性分布，考虑到本题数据量有限，则选择 Spearman 秩相关系数。

使用相关性分析方法后，使用热力图展示化学成分关联度结果。最后，比较高钾、铅钡类别的化学成分关联关系的差异性。

三、模型假设

1. 假设文物风化程度仅与其自身特点有关，不考虑气候、地理环境等自然影响因素以及发掘方式、储存条件等人为影响因素；
2. 假设将因技术或检验方式等原因监测不到的化学成分含量规定为 0 对模型求解的影响微乎其微。
3. 假设文物不会因为缺损等原因而导致化学成分改变，即风化程度为唯一原因。
4. 假设有限的样本数量有足够强的代表性和广泛性，能够体现样本组的普遍规律。

四、符号说明

符号	说明
χ^2	卡方检验的统计量
K	亚类分类数量
A	变量的实际频数
T	变量的理论频数
P	显著性观测值
x'	化学成分含量标准化后的数据
V	有序统计量的协方差
r	相关系数
n	样本量

五、模型建立与求解

5.1 问题一模型的建立与求解

对于本题，解题思路主要分为三个部分，第一部分是利用卡方检验分别分析文物表面风化与其玻璃类型、纹饰和颜色的关系，此处着重分析差异性。第二部分是归纳化学成分的统计规律，结合化学反应知识，从中可以找出文物风化时化

学成分的变化规律，为第三部分建立成分预测模型准备条件。第三部分是化学成分含量的预测问题。

5.1.1 数据的改进和优化

1. 补充缺失数据，将“未检测到”的化学成分记为 0.00。
2. 对已有数据进行整合分类，已风化文物的未分化点记为“无风化”，严重风化点仍记为“风化”，并在作图时对“严重风化”和一般“风化”作出区别。
3. 计算所有化学成分所占比例的总和，保留累加和介于 85%~105%之间的数据，保证数据的有效性。

5.1.2 风化程度与其他属性的关联性检验模型

首先使用堆积直方图，直观展示文物的风化程度与其颜色、类型和纹饰的关系。

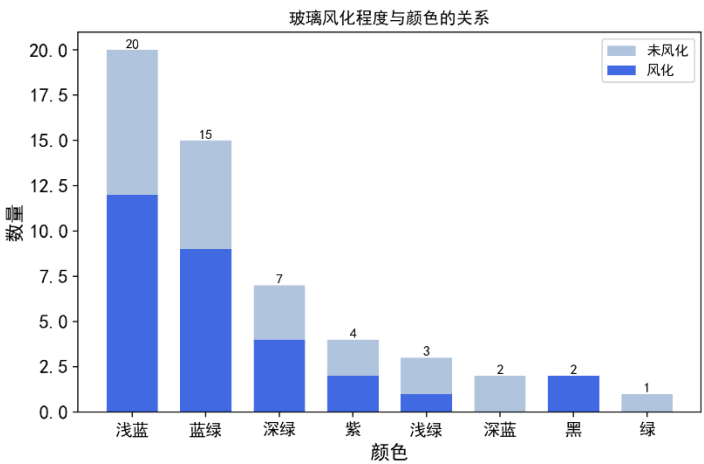


图 1 风化程度与颜色的关系

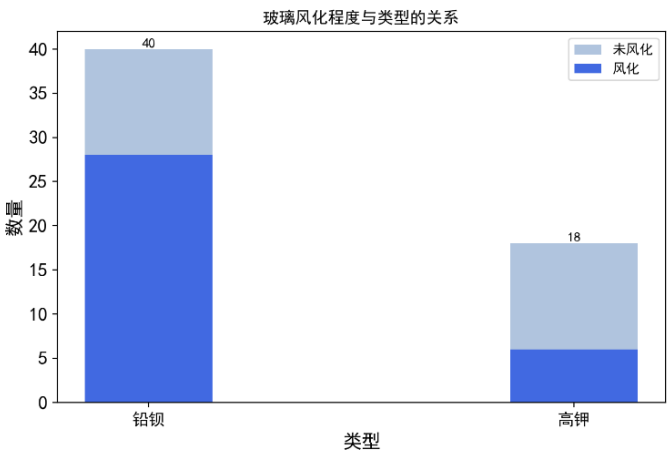


图 2 风化程度与类型的关系

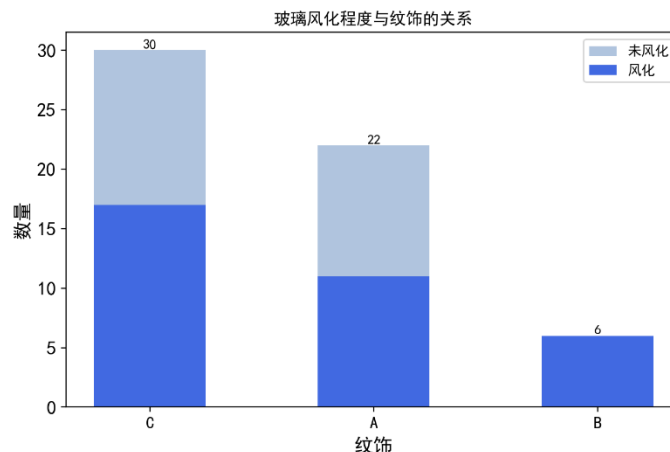


图 3 风化程度与纹饰的关系

由图 1、图 2、图 3 可以观察到玻璃类型不同，风化的比例明显不同，而玻璃的颜色、纹饰对风化的影响较小。B 类型纹饰也会对风化有所影响，但样本量较小。

为量化玻璃表面风化与三种属性的关系，采用差异性分析的方法，比较每个属性（颜色、纹饰、类型）中每个类别（如 A、B、C 纹饰）对表面风化是否造成差异性影响。如不同类别不造成差异性影响，说明该属性与表面风化的关联性弱。由于玻璃的纹饰、类型、颜色与表面风化均属于定类变量，采用卡方检验方法比较定类变量与定类变量之间的差异性。

$$\text{卡方检验公式: } \chi^2 = \sum \frac{(A-T)^2}{T}$$

卡方检验的统计量是卡方值，它是每个格子实际频数 A 与理论频数 T 差值平方与理论频数之比的累计和，表示观察值与理论值之间的偏离程度。

例如：每个格子中的理论频数 T 是在假定三组纹饰的风化率相等（均等于三组合计的风化率）的情况下计算出来的，如下表中无风化——纹饰 C 对应的理论频数为 $24 \times (30/58) = 12.41$ ，实际频数为 13，故卡方值越大，说明实际频数与理论频数的差别越明显，不同组间风化率不同的可能性越大。根据卡方值可得相应的 p 值。

首先分析卡方检验是否呈现显著性，若 $P < 0.05$ 则呈现显著性。具体根据类别的差异百分比进行描述，也可接着根据效应指标对差异进行深入量化分析。

(1) 以 X: {纹饰} 和 Y: {表面风化} 为变量

对 X 和 Y 进行卡方检验分析可得：

表格 1 纹饰和表面风化的卡方检验结果

题目	名称	纹饰			总计	χ^2	校正 χ^2	P
		C	A	B				
表面风化	无风化	13	11	0	24	4.957	4.957	0.084*
	风化	17	11	6	34			

合计	30	22	6	58			
----	----	----	---	----	--	--	--

注：***、**、*分别代表 1%、5%、10%的显著性水平

卡方检验分析的结果显示，基于纹饰和表面风化，显著性 P 值为 0.084*，水平上不呈现显著性，因此对于纹饰和表面风化数据不存在显著性差异。

(2) 以 X: {类型} 和变 Y: {表面风化} 为变量

对 X 和 Y 进行卡方检验分析可得：

表格 2 类型和表面风化的卡方检验结果

题目	名称	类型		总计	X ²	校正 X ²	P
		高钾	铅钡				
表面风化	无风化	12	12	24	6.880	5.452	0.009***
	风化	6	28	34			
合计		18	40	58			

注：***、**、*分别代表 1%、5%、10%的显著性水平

卡方检验分析的结果显示，基于类型和表面风化，显著性 P 值为 0.009***，水平上呈现显著性，拒绝原假设，因此对于类型和表面风化数据存在显著性差异。

(3) 以 X: {颜色} 和变量 Y: {表面风化} 为变量

对 X 和 Y 进行卡方检验分析可得：

表格 3 颜色和表面风化的卡方检验结果

题目	名称	颜色								总计	X ²	校正 X ²	P
		蓝绿	浅蓝	紫	深绿	深蓝	浅绿	绿	黑				
表面风化	无风化	6	8	2	3	2	2	1	0	24	6.287	6.287	0.507
	风化	9	12	2	4	0	1	0	2	30			
合计		15	20	4	7	2	3	1	2	54			

注：***、**、*分别代表 1%、5%、10%的显著性水平

卡方检验用于检验两分组变量是否存在显著性差异：基于颜色和表面风化，显著性 P 值为 0.507，水平上不呈现显著性，接受原假设，因此对于颜色和表面风化数据不存在显著性差异。

由此得出结论，铅钡玻璃更容易发生风化，高钾玻璃相对不容易发生风化。不同颜色的文物间风化程度无明显差异性，不同纹饰的文物间风化程度无明显差异性。

5.1.3 各化学成分的统计规律分析

为了直观准确地研究不同类别玻璃之中的化学成分比例以及其与风化程度的关系，以化学成分为横坐标变量，化学成分的含量为纵坐标变量，以“无风化”、“风化”、“严重风化”为分类绘制统计规律折线图。

之后分别以风化铅钡玻璃、未风化铅钡玻璃、未风化高钾玻璃、风化高钾玻璃为样本组，计算十四种化学成分含量所占比例的最大值、最小值、平均值、标准差。

数字 0-12 代表的化学成分如下所示

表格 4 数值代表的化学成分

0	1	2	3	4	5	6	7	8	9	10	11	12	13
二氧化硅(SiO2)	氧化钠(Na2O)	氧化钾(K2O)	氧化钙(CaO)	氧化镁(MgO)	氧化铝(Al2O3)	氧化铁(Fe2O3)	氧化铜(CuO)	氧化铅(PbO)	氧化钡(BaO)	五氧化二磷(P2O5)	氧化锶(SrO)	氧化锡(SnO2)	二氧化硫(SO2)

(1) 铅钡玻璃化学成分的统计规律

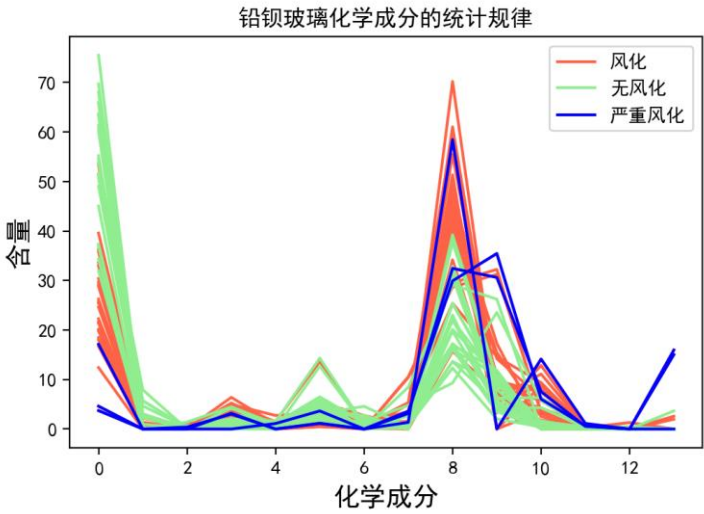


图 4 铅钡玻璃化学成分的统计规律（单位%）

表格 5 铅钡玻璃化学成分的统计规律

变量名	无风化（样本量 23）				风化（样本量 26）			
	最大值	最小值	平均值	标准差	最大值	最小值	平均值	标准差
二氧化硅(SiO2)	75.51	31.94	54.66	11.829	53.33	3.72	24.913	10.605
氧化钠(Na2O)	7.92	0	1.683	2.372	2.22	0	0.216	0.557
氧化钾(K2O)	1.41	0	0.219	0.31	1.05	0	0.133	0.24
氧化钙(CaO)	4.49	0	1.32	1.285	6.4	0	2.695	1.66
氧化镁(MgO)	1.67	0	0.64	0.547	2.73	0	0.65	0.706
氧化铝(Al2O3)	14.34	1.42	4.456	3.262	13.65	0.45	2.97	2.634
氧化铁(Fe2O3)	4.59	0	0.737	1.155	2.74	0	0.585	0.737
氧化铜(CuO)	8.46	0	1.432	1.97	10.57	0	2.276	2.821
氧化铅(PbO)	39.22	9.3	22.085	8.215	70.21	15.71	43.314	12.23
氧化钡(BaO)	26.23	2.03	9.002	5.825	35.45	0	11.807	9.978
五氧化二磷(P2O5)	6.34	0	1.049	1.847	14.13	0	5.277	4.197
氧化锶(SrO)	0.91	0	0.268	0.243	1.12	0	0.418	0.265

氧化锡(SnO ₂)	0.44	0	0.047	0.127	1.31	0	0.068	0.269
二氧化硫(SO ₂)	3.66	0	0.159	0.763	15.95	0	1.366	4.206

由图表所示，可得以下结论：

1. 铅钡玻璃风化后二氧化硅含量大幅减小，而氧化铅含量大幅增加，氧化钡有一定程度增加，五氧化二磷、氧化锶、氧化锡、二氧化硫等也有一定程度增加。
2. 风化程度越高，二氧化硅含量越少。
3. 风化过后氧化钠的含量减少

(2) 高钾玻璃化学成分的统计规律

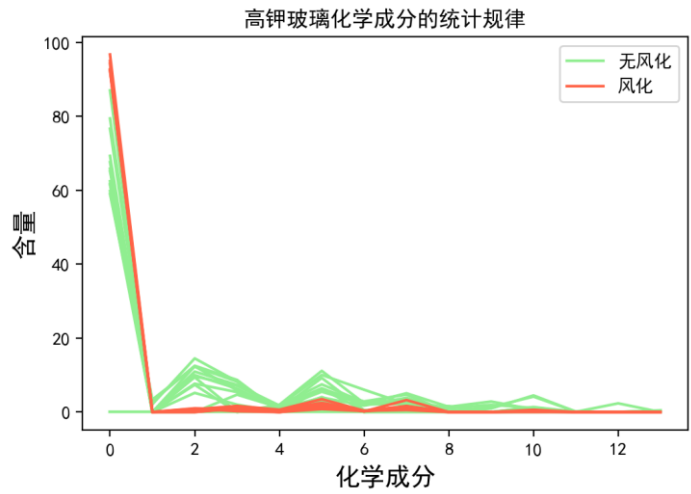


图 5 高钾玻璃化学成分的统计规律（单位%）

表格 6 高钾玻璃化学成分的统计规律

变量名	风化（样本量 6）				无风化（样本量 12）			
	最大值	最小值	平均值	标准差	最大值	最小值	平均值	标准差
二氧化硅(SiO ₂)	96.77	92.35	93.963	1.734	87.05	59.01	67.984	8.755
氧化钠(Na ₂ O)	0	0	0	0	3.38	0	0.695	1.287
氧化钾(K ₂ O)	1.01	0	0.543	0.445	14.52	0	9.331	3.92
氧化钙(CaO)	1.66	0.21	0.87	0.488	8.7	0	5.333	3.092
氧化镁(MgO)	0.64	0	0.197	0.306	1.98	0	1.079	0.676
氧化铝(Al ₂ O ₃)	3.5	0.81	1.93	0.964	11.15	3.05	6.62	2.492
氧化铁(Fe ₂ O ₃)	0.35	0.17	0.265	0.069	6.04	0	1.932	1.667
氧化铜(CuO)	3.24	0.55	1.562	0.935	5.09	0	2.452	1.66
氧化铅(PbO)	0	0	0	0	1.62	0	0.412	0.589
氧化钡(BaO)	0	0	0	0	2.86	0	0.598	0.982
五氧化二磷(P ₂ O ₅)	0.61	0	0.28	0.21	4.5	0	1.402	1.434

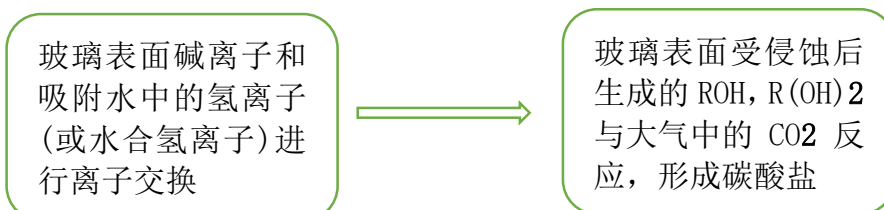
氧化锶(SrO)	0	0	0	0	0.12	0	0.042	0.048
氧化锡(SnO ₂)	0	0	0	0	2.36	0	0.197	0.681
二氧化硫(SO ₂)	0	0	0	0	0.47	0	0.102	0.186

由图表所示，可得以下结论：

1. 高钾玻璃风化后二氧化硅含量大幅增加，其余所有成分都有一定程度减少，其中氧化铝和氧化钾尤为明显。
2. 高钾玻璃风化后二氧化硅含量的标准差显著减少，且氧化钠、氧化铅、氧化钡、氧化锶、氧化锡、二氧化硫等氧化物消失。
3. 风化过后氧化钠的含量减少
4. 方差分析（这里特指单因素方差分析），其默认有个前提条件是因变量需要符合正态分布性，可能是由于即使非正态分布，方差分析的结果也较为稳健，因而很少有人先进行正态分布性检验。

综合以上两个图示、表格得到以下猜想：

1. 铅钡玻璃更容易发生风化，高钾玻璃相对不容易发生风化的原因是铅钡玻璃含过氧化钠的含量较高
2. 风化的本质是二氧化硅与周围环境或者文物本身所含金属元素间的氧化还原反应。
3. 氧化钠相当于反应中催化剂的作用，反应原理如下：



最后使用单因素方差分析法，检验不同分组数据是否存在显著性差异，并找出在风化前后变化最大的化学成分，即统计规律中对有无风化影响最大的化学成分。各化学成分风化前后均值图如下，通过比较均值，可以挖掘其差异关系：

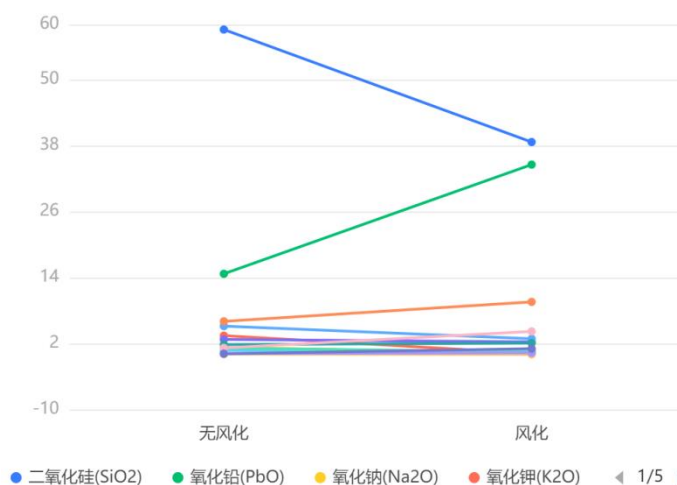


图 6 风化前后均值图

结合风化前后均值图，选择部分化学成分计算方差、标准差，并得出方差分析结果 P 值。

表格 7 方差分析结果表

变量名	变量值	样本量	平均值	标准差	P
二氧化硅 (SiO ₂)	无风化	34	59.098	12.674	0.000***
	风化	33	38.642	28.894	
	总计	67	49.022	24.316	
氧化钠 (Na ₂ O)	无风化	34	1.294	2.108	0.010***
	风化	33	0.262	0.706	
	总计	67	0.786	1.653	
氧化钾 (K ₂ O)	无风化	34	3.438	4.97	0.000***
	风化	33	0.207	0.319	
	总计	67	1.847	3.879	
氧化铝 (Al ₂ O ₃)	无风化	34	5.173	3.203	0.002***
	风化	33	2.875	2.461	
	总计	67	4.041	3.067	
氧化铁 (Fe ₂ O ₃)	无风化	34	1.18	1.454	0.019**
	风化	33	0.509	0.67	
	总计	67	0.849	1.179	
氧化铅 (PbO)	无风化	34	14.683	12.543	0.000***
	风化	33	34.54	20.414	
	总计	67	24.463	19.513	
氧化锶(SrO)	无风化	34	0.188	0.229	0.021**
	风化	33	0.338	0.286	
	总计	67	0.262	0.267	
氧化钡 (BaO)	无风化	34	6.036	6.29	0.086*
	风化	33	9.575	9.952	
	总计	67	7.779	8.425	
五氧化二磷 (P ₂ O ₅)	无风化	34	1.205	1.717	0.000***
	风化	33	4.209	4.259	
	总计	67	2.684	3.544	

注: ***, **, *分别代表1%、5%、10%的显著性水平

根据图表,可得出二氧化硅、氧化钠、氧化钾、氧化铝、氧化铅的方差分析结果 P 远小于 0.05, 统计结果显著, 说明不同的表面风化在这些化学成分存在存在显著差异。

5.1.4 基于二氧化硅与金属元素间转换关系的化学成分预测模型

岭回归是一种改良的最小二乘估计法,通过放弃最小二乘法的无偏性,以损失部分信息、降低精度为代价获得回归系数更为符合实际、更可靠的回归方法,对于该样本数较少,数据量较大的数据拟合度要强于最小二乘法。

根据 5.1.3 推测,将二氧化硅作为因变量,其他碱金属和表面风化等元素作为自变量,分别对高钾和铅钡玻璃进行建模。

高钾模型预测图如下,与样本数据基本吻合,该模型的拟合优度 R^2 为 0.998,,模型表现为较为优秀,因此模型基本满足要求。

模型的具体公式为: 二氧化硅(SiO₂)=91.055-2.102 × 氧化钠(Na₂O)-0.651 × 氧化钾(K₂O)-0.898 × 氧化钙(CaO)-2.189 × 氧化镁(MgO)-0.624 × 氧化铝(Al₂O₃)-1.074 × 氧化铁(Fe₂O₃)-1.103 × 氧化铜(CuO)-0.756 × 氧化铅(PbO)-0.409 × 氧化钡(BaO)-0.476 × 五氧化二磷(P₂O₅)-26.878 × 氧化锶(SrO)-0.546 × 氧化锡(SnO₂)-6.154 × 二氧化硫(SO₂)+3.756 × 表面风化

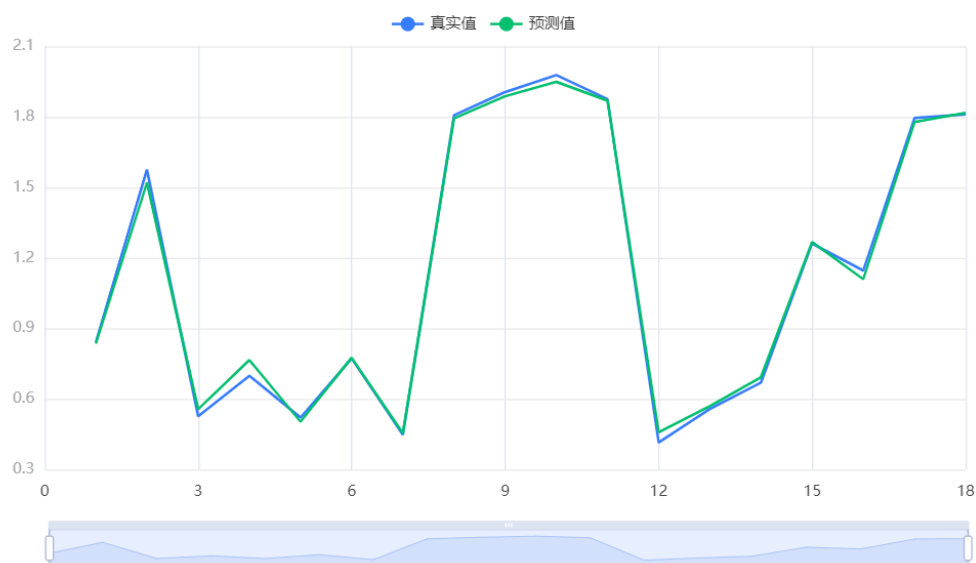


图 7 高钾预测效果图

高钾模型预测图如下，与样本数据基本吻合，该模型的拟合优度 R^2 为 0.955，模型表现为较为优秀，因此模型基本满足要求。

模型的具体公式为：

二氧化硅 (SiO_2) = $85.917 - 0.426 \times \text{氧化钠} (\text{Na}_2\text{O}) - 3.032 \times \text{氧化钾} (\text{K}_2\text{O}) - 1.903 \times \text{氧化钙} (\text{CaO}) + 0.543 \times \text{氧化镁} (\text{MgO}) + 0.14 \times \text{氧化铝} (\text{Al}_2\text{O}_3) - 0.097 \times \text{氧化铁} (\text{Fe}_2\text{O}_3) - 0.598 \times \text{氧化铜} (\text{CuO}) - 0.516 \times \text{氧化铅} (\text{PbO}) - 0.66 \times \text{氧化钡} (\text{BaO}) - 0.814 \times \text{五氧化二磷} (\text{P}_2\text{O}_5) - 6.864 \times \text{氧化锶} (\text{SrO}) + 0.756 \times \text{氧化锡} (\text{SnO}_2) - 0.484 \times \text{二氧化硫} (\text{SO}_2) - 8.073 \times \text{表面风化}$

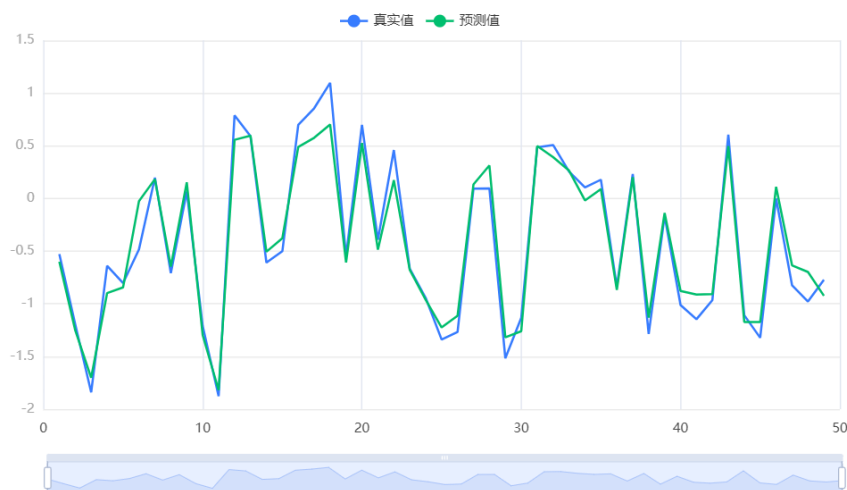


图 8 铅钡预测效果图

5.2 问题二模型的建立与求解

问题二首先需要分析两种玻璃的分类规律，考虑到风化后玻璃化学成分比例大幅变化，可能对类别分析产生影响，故先根据第一问公式预测已风化玻璃风化

前的化学成分后，用随机森林分类器分析对玻璃分类产生主要影响的特征化学元素。

问题二的难点在于要分别对两种玻璃进行聚类分析。聚类与分类的不同在于，聚类所要求划分的类是未知的，分类标准和分类数量都要通过分析求得。针对问题二，可分为以下步骤：

1. 首先可建立基于 Kmeans 算法的高维聚类分析模型，并用 Z-score 标准化和 LOF 算法对聚类模型进行了创新优化。

2. 其次，Kmeans 算法可以通过从 1 到样本数的循环，根据计算得出的轮廓系数值，衡量聚类效果的好坏，给出最佳的 K 值选择。考虑到玻璃文物样本有限，亚类数量不宜超过 3 个。

3. 然后按照第一小问思路，通过基于随机森林算法的重要度分析确定影响聚类模型的主要化学成分，并按此指定亚类分类标准；为直观可视化亚类分布，最后对文物化学组成的多维特征进行主成分分析降维，最终得到样本亚类分布的散点图。

4. 分析模型合理性和灵敏性。

5.2.0 随机森林显著变量选择

风化与无风化样品对玻璃的分类影响较大，为了降低该因素对模型稳定性的影响，考虑将风化后的化学成分数据按照 5.1 中预测公式全部还原为风化前数据，再进行建模分析。

选择随机森林模型分析 14 种化学成分即特征值对玻璃类型分类的重要性，得到下图

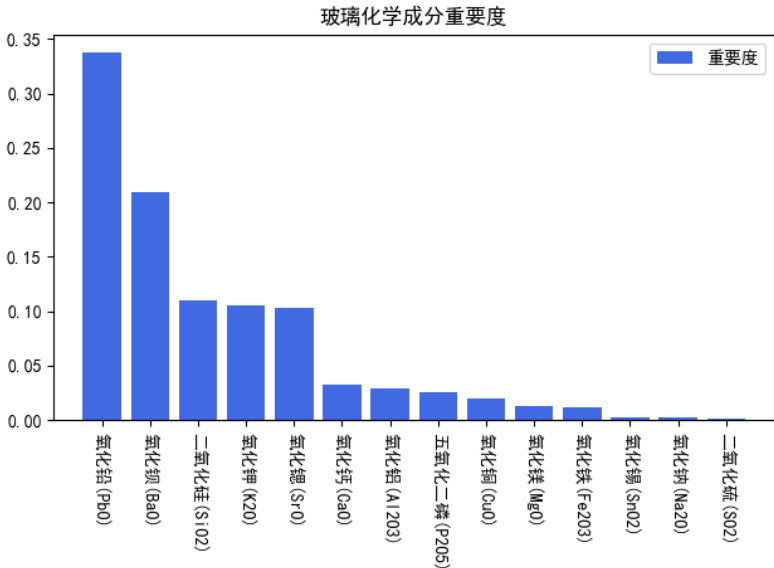


图 9 不同类型玻璃化学成分重要度

可看到铅钡元素对玻璃分类影响很大，实际上根据 5.1 知铅钡玻璃铅钡元素含量高，高钾玻璃钾元素含量高，与随机森林预测基本符合。

5.2.1 无监督聚类算法——Kmeans

K- Means 是迭代动态聚类算法中的一种，其中 K 表示类别数，Means 表示各类别数据的均值。Kmeans 算法解决的问题是，在事先不知道任何分类的情况下，通过从 1 到样本数量的循环，迭代出最佳分类方案。Kmeans 的运行逻辑如下：

1. 随机选取数据集中的 k 个点作为初始点。
2. 根据相似性度量函数，将其他点与初始点比较，选择距离近的归类
3. 将 N 个对象最优地划分为 K 个分类。

在此题中，拟通过 python 编程求解。

5.2.2 基于数据预处理的聚类模型改进

在聚类分析中，数据样本的微小偏差也会对聚类产生比较大的影响。考虑到玻璃文物的样本数量有限，个别极端值会对模型产生较大的影响，例如在已风化铅钡玻璃的样本中二氧化硅(SiO₂)含量的最小值低至 3.72%，就属于严重风化情况下的一个极端数值。为了获得更完善的模型，在原有高维聚类模型的基础上加以数据预处理改进。数据预处理包括选择数量，类型和特征的标度，主要是清理异常值、纠正极端数据。

1. Z-score 标准化

标准化指将数据按比例缩放，使之落入一个小的特定区间，以减少规模、特征、分布差异等对数据模型的影响。

Z-score 标准化公式：

$$x' = \frac{x - \text{mean}}{\text{std}}$$

此处 mean 为每种化学成分含量的数据均值，std 为它的标准差。转化之后的数据符合均值为 0，方差为 1 的正态分布。有利于减轻极端数据对整体准确性的影响。

2. LOF 算法去除异常值

LOF 算法即局部离群因子检测方法，是一种无监督的离群检测方法。可以去除个别特殊的异常数据。

高钾玻璃和铅钡玻璃的每种化学成分的含量数据经过预处理后，基于 LOF 算法用 python 编程处理，得到可供聚类模型分析的有效样本组。

（注：这一点在敏感性测试中也有用处，保证数据有效）

5.2.3 基于轮廓系数的亚类数量确定

K-means 算法将样本分为 K 类，对于每一个 K 值，都可以通过求出其所对应的轮廓系数来评价分类效果的好坏。理论上轮廓系数越趋近于 1，对应的 K 值聚类效果就越好。

利用 python 编程得到两组玻璃样本 K 值与轮廓系数的关系。如下图所示

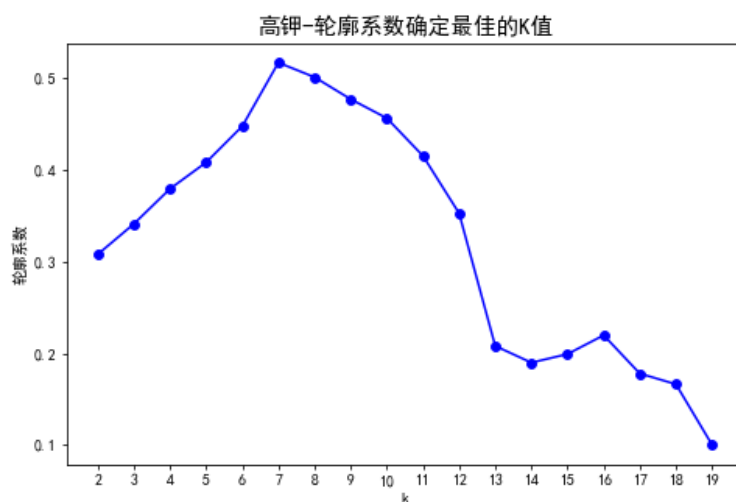


图 10 高钾-轮廓系数确定最佳的 K 值

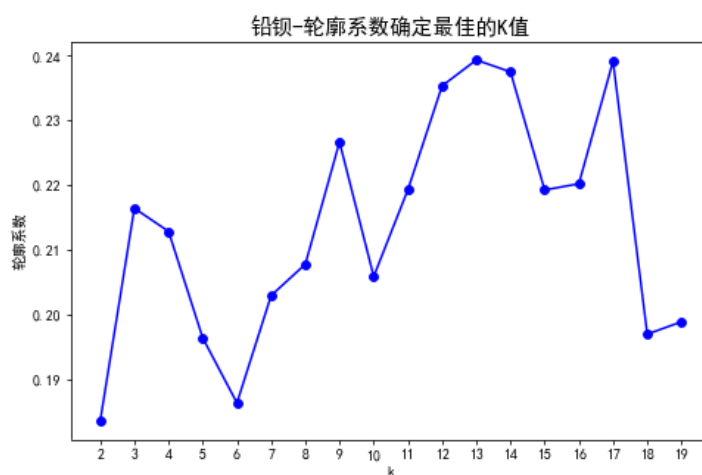


图 11 铅钡-轮廓系数确定最佳的 K 值

5.2.4.基于随机森林算法的重要度分析

1. 随机森林分类器

针对问题二的以随机森林法查看分类后主成分，以此决定分类标准(合理性)。每个特征在随机森林中的每颗树上做出的贡献，取个平均值，决定特征的贡献大小。文物的 14 个化学成分，就可以等价于 14 个特征。但是每个化学成分对于该文物成为某种类别的贡献度不同。评估各个成分的贡献程度即对所有成分进行重要度检验，重要度最高的成分就是最典型的特征。

2. 聚类后数据初步分析

高钾聚类分析为 2 个亚类，统计其每个亚类标准差与平均值如下：

表格 8 高钾聚类分析

	聚类类别（平均值±标准差）	
	类别 1(n=10)	类别 2(n=8)

二氧化硅(SiO ₂)	0.659±0.22	1.751±0.23
氧化钠(Na ₂ O)	0.029±0.839	-0.479±0.0
氧化钾(K ₂ O)	2.049±1.061	0.101±0.87
氧化钙(CaO)	1.589±1.102	-0.706±0.295
氧化镁(MgO)	0.711±0.983	-0.533±0.853
氧化铝(Al ₂ O ₃)	1.049±0.736	-0.56±0.375
氧化铁(Fe ₂ O ₃)	1.255±1.324	-0.556±0.116
氧化铜(CuO)	0.409±0.666	-0.308±0.441
氧化铅(PbO)	-1.239±0.033	-1.262±0.005
氧化钡(BaO)	-0.844±0.124	-0.93±0.0
五氧化二磷(P ₂ O ₅)	-0.342±0.445	-0.632±0.121
氧化锶(SrO)	-0.825±0.191	-0.954±0.093
氧化锡(SnO ₂)	-0.232±0.0	0.649±2.493
二氧化硫(SO ₂)	-0.18±0.074	-0.225±0.0

铅钡聚类分析为 3 个亚类，统计其每个亚类标准差与平均值如下

表格 9 铅钡聚类分析

	聚类类别（平均值±标准差）		
	类别 1(n=25)	类别 2(n=19)	类别 3(n=5)
二氧化硅(SiO ₂)	0.165±0.551	-0.942±0.349	-1.367±0.493
氧化钠(Na ₂ O)	0.572±1.374	-0.44±0.171	-0.479±0.0
氧化钾(K ₂ O)	-0.435±0.043	-0.428±0.102	-0.459±0.046
氧化钙(CaO)	-0.66±0.345	0.4±0.67	-0.266±0.5
氧化镁(MgO)	-0.176±0.936	0.36±0.944	-1.058±0.0
氧化铝(Al ₂ O ₃)	0.144±1.24	-0.259±0.501	-0.938±0.108
氧化铁(Fe ₂ O ₃)	-0.241±0.841	0.081±0.806	-0.726±0.0
氧化铜(CuO)	-0.363±0.526	-0.233±0.724	2.374±1.631
氧化铅(PbO)	-0.002±0.61	1.111±0.583	0.283±0.076
氧化钡(BaO)	0.144±0.576	-0.089±0.498	2.796±0.398
五氧化二磷(P ₂ O ₅)	-0.527±0.406	1.034±1.204	0.4±0.812
氧化锶(SrO)	-0.188±0.742	0.772±0.989	1.184±0.786
氧化锡(SnO ₂)	-0.048±0.789	-0.026±0.49	-0.232±0.0
二氧化硫(SO ₂)	-0.171±0.273	-0.225±0.0	2.428±2.884

3. 重要度分析

利用 python 环境下的 sklearn 包得到高钾和铅钡玻璃中化学元素的重要度。其中重要度的分类标准为：主成分平均值±标准差

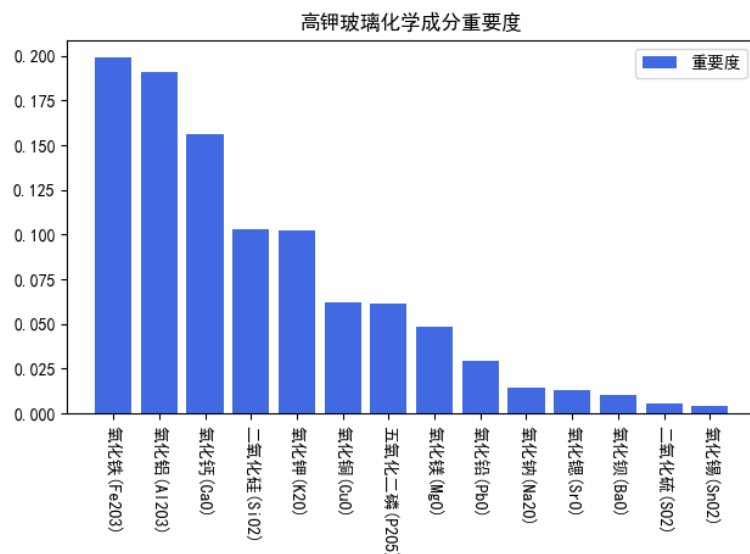


图 12 高钾玻璃化学成分重要度

由上图可得：选择重要度最高的铁元素作为亚类划分，当铁元素含量在 0.22 ± 0.15 范围间，为高铁亚类，在 2.75 ± 1.36 时为高铁亚类。

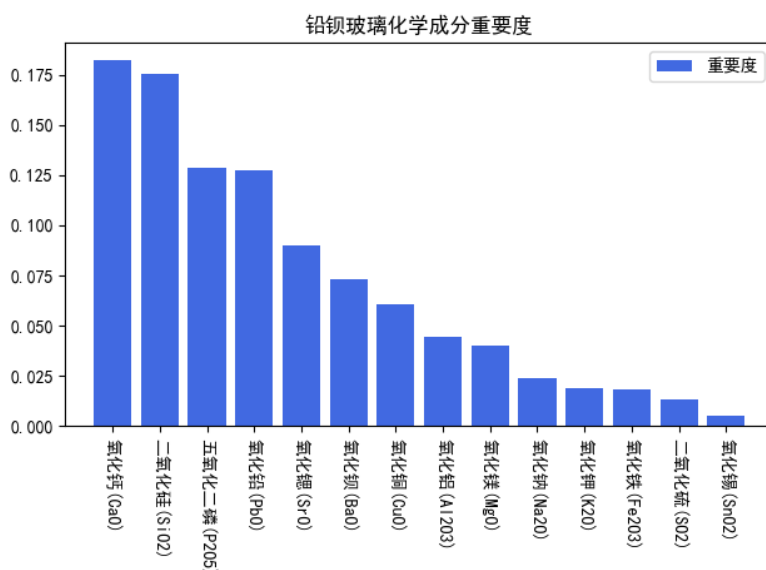


图 13 铅钡玻璃化学成分重要度

由上图可得：选择重要度最高的钙元素作为亚类划分，当钙元素含量在 0.72 ± 0.455 分为低钙亚类，在 3.03 ± 0.83 时为中钙亚类，当该元素含量在 5.43 ± 0.66 时为高钙亚类。

5.2.5 由主成分分析降维模型得出划分结果

玻璃文物的分类主要依据其化学成分判定，而可检测到的成分足足有 14 种，每种成分所占重要度又各不相同。此时需要将多个变量通过线性变换以选出较少

个数重要变量，即从原始变量中导出少数几个主成分，使它们尽可能多地保留原始变量的信息，将原来 P 个指标作线性组合，作为新的综合指标。此处用到的这种方法称作主成分分析。

高维聚类分析经过 PCA 降维的过程如下：

1. PCA 分解： $X=TP^T \longrightarrow XP=T$
2. T 是 X 在新的主成分空间中的样本新坐标。 T 的列的安排，分别是散度第一、第二...，最大的方向。取 T 的前 2 列制图，可观察样本在分离度最大方向的聚集情况。
3. 选取主成分数目。 n 维数据集可以通过映射降成 k 维子空间，其中 k 是选取的主成分数目。PCA 分别将高钾和铅钡的每个文物的 14 条成分数据投射到一个低维子空间实现降维。
- 4.降维分类。根据样本在空间中聚集模式，达到分类的目的。空间维数太高，寻找空间的散度最大方向的超平面。超平面形成 2 维空间，在人可辨别的低维空间完成分类

绘图显示分类结果：

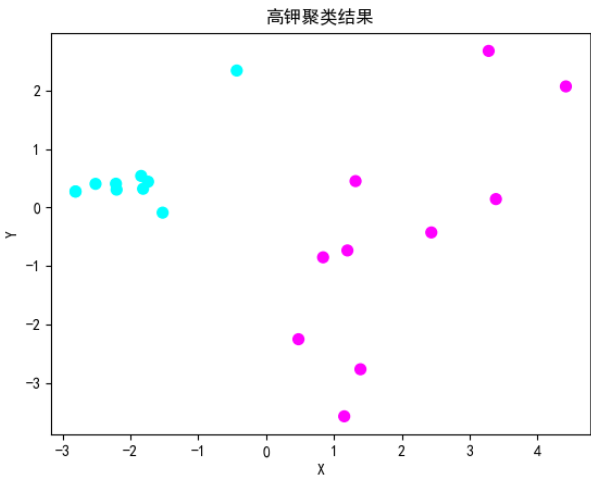


图 14 高钾聚类结果

高钾玻璃聚类结果为：分为两个亚类

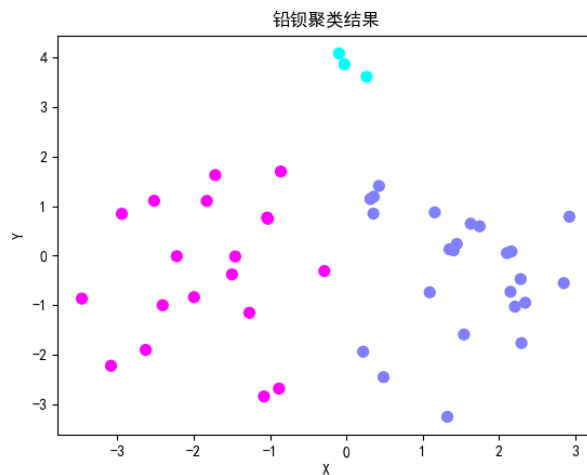


图 15 铅钡聚类结果
铅钡玻璃的聚类结果为：分为三个亚类

5.2.6 合理性和敏感性检验

根据聚类结果图，类别之间区分明显，集聚性好，分类效果较显著，说明模型有一定合理性，同时在建模过程中，轮廓系数确定合适 k 值与随机森林选取主成分等也保障了模型建立的合理性。

Kmeans 模型的数据经过标准化处理和异常值去除，在增加一些干扰性数据进行测试时，会先进行筛选删除，模型变化不明显，保障了模型的灵敏度。同时，在调节 kmeans 参数时，修改模型误差值、增加计算层数等都对最后分类影响不明显，说明模型具有一定的稳定性。

5.3 问题三模型的建立与求解

考虑到高钾和铅钡风化前后差异较大，将训练数据分为风化与无风化两类进行分别建模，同时测试数据按表面风化分为风化与无风化类型分别进行预测。该样本数据较少且作为变量的化学成分较多且具有一定相关性，选择对变量空间进行降维并回归的方法进行建模，此处主要用到了主成分回归（PCR）与偏最小二乘回归（PLS）的方法建立预测模型并进行对比分析。

5.3 问题三模型的建立与求解

考虑到高钾和铅钡风化前后差异较大，将训练数据分为风化与无风化两类进行分别建模，同时测试数据按表面风化分为风化与无风化类型分别进行预测。该样本数据较少且作为变量的化学成分较多且具有一定相关性，选择对变量空间进行降维并回归的方法进行建模，此处主要用到了主成分回归（PCR）与偏最小二乘回归（PLS）的方法建立预测模型并进行对比分析。

5.3.1 数据预处理

在具体建模之前，先对数据进行标准化处理，其训练集处理方式同 5.2，而测试集按照训练集的标准差与平均值进行标准化。在对类别数据进行处理时，将“高钾”和“铅钒”两种类型转换为 one-hot 编码，“高钾”为[1,0]，“铅钒”为[0,1]，方便模型进行后续预测分析。

5.3.2 K-FOLD 交叉验证

该数据集较小，采用 K-Fold 交叉验证将样本随机分为 k 份，每次选择 k-1 份作为测试集，剩下的 1 份用作测试集进行建模并预测，如此循环若干轮检测后，通过测试集预测的误差率和 F1 得分评估各模型。

误差率计算为：预测错误个数/样本总个数

F1-SCORE 计算为：

（正类：铅钒 负类：高钾）

TP：样本为正类且预测为正类

TN：样本为负类且预测为负类

FP：样本为负类但预测为正类

FN：样本为正类但预测为负类

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

5.3.3 PCR 主成分回归

样本数据含 14 个化学成分，即 14 个自变量，有效样本数仅有 67 个，分为风化和无风化后分别为 32 与 35 个，数据量较小，自变量较多，首先同 5.2 中进行 PCA 降维，消除变量之间的共线关系，并提取主成分变量依此作为自变量进行建模分析。在预测测试集时同样也需将预测数据变量转换到主成分空间进行预测，其预测结果为 softmax 矩阵，选择概率大的一方作为最后预测类别。最后，根据 pca 特征值比值选择合适主成分，风化数据主成分为 4，无风化数据主成分为 10。PCR 建模后进行交叉验证分析后数据如下：

表格 10 PCR 交叉验证数据

	误差率	F1-SCORE
无风化	5.71%	93.66%
风化	0	1

可以看到该模型误差较小，效果不错。

使用该模型预测未知样本类型如下：

表格 11 PCR 预测样本

	表面风化	高钾概率	铅钡概率	预测类型
A1	无风化	2.93e-92	1.00e+00	铅钡
A2	风化	9.99e-01	8.04e-04	高钾
A3	无风化	9.99e-01	1.19e-04	高钾
A4	无风化	2.22e-03	9.98e-01	铅钡
A5	风化	3.45e-01	6.55e-01	铅钡
A6	风化	1.00e+00	1.99e-15	高钾
A7	风化	1.00e+00	2.91e-12	高钾
A8	无风化	2.21e-03	9.98e-01	铅钡

5.3.4 PLS 偏最小二乘回归

基本过程与 PCR 相同，在进行主成分分析时，根据主成分与误差之间的关系确定合适的主成分数，如下

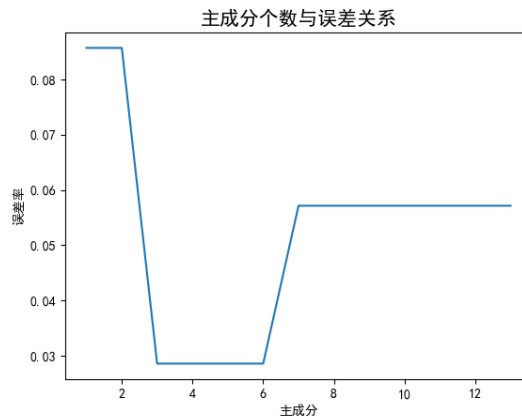


图 16 pls 主成分与误差关系

以此选取主成分为 3，进行 pls 建模，得到交叉验证分析数据如下：

表格 12 PLS 交叉验证

	误差率	F1-SCORE
无风化	2.86%	96.76%
风化	0	1

对比 PCR 模型，其无风化模型较 PCR 误差更小，F1 得分更好，模型更加优秀。以此模型预测未知样本类型如下：

表格 13 PLS 预测样本

	表面风化	高钾概率	铅钡概率	预测类型
A1	无风化	8.17e-01	1.83e-01	高钾
A2	风化	7.08e-01	2.92e-01	高钾
A3	无风化	7.62e-08	9.99e-01	铅钡
A4	无风化	4.39e-04	9.99e-01	铅钡
A5	风化	1.91e-17	1.00e+00	铅钡
A6	风化	9.99e-01	6.05e-04	高钾
A7	风化	9.96e-01	3.71e-03	高钾

A8	无风化	3.03e-05	9.99e-01	铅钡
----	-----	----------	----------	----

PLS 预测无风化样本类型与 PCR 略有区别，而预测风化类型别无二致，考虑到 PLS 模型较优，PLS 预测可信度更高。

5.4 问题四模型的建立与求解

首先针对高钾、铅钡类别的玻璃文物样品，分别分析每个类别的化学成分之间的关联关系。自然界的化学成分之间不存在确定的函数关系，因此需要求得化学成分之间的统计关系，即度量化学成分变量之间的相关性，并用适当的统计指标进行展示。在选择相关性分析方法之前，对数据进行正态性检验。若数据近似满足正态性分布，则选择 Pearson 相关系数；。若数据很难满足正态性分布，考虑到本题数据量有限，应选择 Spearman 秩相关系数。使用相关性分析方法后，使用热力图展示化学成分关联度结果。

最后，比较高钾、铅钡类别的化学成分关联关系的差异性。

5.4.1 正态性分布

在选择相关性分析方法之前，对铅钡、高钾的化学成分数据分布进行正态性检验。根据正态性检验的规定，当样本含量 $n \leq 2000$ 时，结果以 Shapiro-Wilk (W 检验) 为准，当样本含量 $n > 2000$ 时，结果以 Kolmogorov—Smimov (D 检验，修正后的 KS 检验) 为准。由于本题数据量小（铅钡样本量 49，高钾样本量 18），选择 Shapiro-Wilk 作为正态检验方法。计算后得到的相关系数 s 接近 1，则表示数据和正态分布拟合得好。

$$W = \frac{\left(\sum_{i=1}^n a_i x_{(i)} \right)^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

其中

1. $x_{(i)}$ 用括号包含下标索引的；不与 x 混淆，它是第 i 阶统计量，即样本中的第 i 个最小数
2. $\bar{x} = (x_1 + \cdots + x_n) / n$ 是样本的平均值。
3. 常量 $a_i a_i$ 通过公式[1]

$$(a_1, \dots, a_n) = \frac{m^* V^{-1}}{(m^* V^{-1} V^{-1} m)^{1/2}}, m = (m_1, \dots, m_n)^*$$

其中 $m_1, \dots, m_n$ 是从一个标准的正态分布随机变量上采样的有序独立同分布的统计量的期望值。

V 是这些有序统计量的协方差。

首先对铅钡玻璃的化学成分进行正态性检验。这个统计检验的假设是样本来自于一个正态母体，因此如果 p 值小于选择的显著度水平 0.05，那么在更大概率下应该拒绝零假设，表示样本不是来自一个正态分布母体。

表格 14 铅钡玻璃化学成分正态性检验

铅钡	二氧化硅 (SiO ₂)	氧化钠 (Na ₂ O)	氧化钾 (K ₂ O)	氧化钙 (CaO)	氧化镁 (MgO)	氧化铝 (Al ₂ O ₃)	氧化铁 (Fe ₂ O ₃)	氧化铜 (CuO)	氧化铅 (PbO)	氧化钡 (BaO)	五氧化二磷 (P ₂ O ₅)	氧化锶 (SrO)	氧化锡 (SnO ₂)	二氧化硫 (SO ₂)
s	0.96077889	0.5795735	0.654394	0.920306	0.865228	0.761021	0.732101	0.691709	0.962156	0.815018	0.81632	0.933597	0.306533	0.275315
p	0.10199666	1.283E-10	1.76E-09	0.002714	4.82E-05	1.53E-07	4.09E-08	7.47E-09	0.116212	2.39E-06	2.57E-06	0.008383	7.4E-14	3.64E-14

根据显著性 p 值可得，不符合正态分布的化学成分有：

- 1.氧化钠(Na₂O)
- 2.氧化钾(K₂O)
- 3.氧化钙(CaO)
- 4.氧化镁(MgO)
- 5.氧化铝(Al₂O₃)
- 6.氧化铁(Fe₂O₃)
- 7.氧化铜(CuO)
- 8.氧化钡(BaO)
- 9.五氧化二磷(P₂O₅)
- 10.氧化锶(SrO)
- 11.氧化锡(SnO₂)
- 12.二氧化硫(SO₂)

之后对高钾玻璃的化学成分进行正态性检验。

表格 15 高钾玻璃化学成分正态性检验

高钾	二氧化硅 (SiO ₂)	氧化钠 (Na ₂ O)	氧化钾 (K ₂ O)	氧化钙 (CaO)	氧化镁 (MgO)	氧化铝 (Al ₂ O ₃)	氧化铁 (Fe ₂ O ₃)	氧化铜 (CuO)	氧化铅 (PbO)	氧化钡 (BaO)	五氧化二磷 (P ₂ O ₅)	氧化锶 (SrO)	氧化锡 (SnO ₂)	二氧化硫 (SO ₂)
s	0.8519721	0.4823291	0.864074	0.859309	0.881701	0.948025	0.788019	0.948931	0.605195	0.55568	0.692481	0.673424	0.252692	0.478593
p	0.00909877	5.652E-07	0.014231	0.011916	0.027884	0.394878	0.001034	0.408496	7.62E-06	2.54E-06	6.54E-05	3.98E-05	1.06E-08	5.25E-07

根据显著性 p 值可得，不符合正态分布的化学成分有：

- 1.二氧化硅(SiO₂)
- 2.氧化钠(Na₂O)
- 3.氧化钾(K₂O)
- 4.氧化钙(CaO)
- 5.氧化镁(MgO)
- 6.氧化铁(Fe₂O₃)
- 7.氧化铅(PbO)
- 8.氧化钡(BaO)
- 9.五氧化二磷(P₂O₅)
- 10.氧化锶(SrO)
- 11.氧化锡(SnO₂)
- 12.二氧化硫(SO₂)

因此，考虑到本题数据量有限，且数据很难满足正态性分布，应选择 Spearman 秩相关系数。

5.4.2 Spearman 秩相关系数热力图

Spearman 秩相关系数是衡量两个变量之间相关性的非参数指标。它利用单调方程评价两个统计变量的相关性，如果数据中没有重复值，并且当两个变量完全单调正相关时，斯皮尔曼相关系数则为+1，负相关则为-1。

ρ = (Σ (xi - x̄)(yi - ȳ)) / (sqrt(Σ (xi - x̄)² Σ (yi - ȳ)²))

首先计算铅钡玻璃的 Spearman 秩相关系数。在计算相关系数之前，需要对化学成分之间是否存在统计意义上的显著性关系进行检验，判断 P 值是否有显著性。一般地，判断 P 是否小于 0.05，即在 95%的置信水平上拒绝原假设，样本有相关性较为显著。

表格 16 铅钡 Spearman 显著性 P 值

	二氧化硅(SiO2)	氧化钠(Na2O)	氧化钾(K2O)	氧化钙(CaO)	氧化镁(MgO)	氧化铝(Al2O3)	氧化铁(Fe2O3)	氧化铜(CuO)	氧化铅(PbO)	氧化钡(BaO)	五氧化二磷(P2O5)	氧化锶(SrO)	氧化锆(SnO2)	二氧化硫(SO2)
二氧化硅(SiO2)	1	0.003912207	0.031225862	0.000353755	0.399575589	0.002356357	0.764996542	0.001289055	3.02276E-10	0.051714406	6.74148E-05	3.86712E-05	0.501692522	0.028332439
氧化钠(Na2O)	0.003912207	1	0.648713159	0.009517588	0.873151433	0.319754929	0.158818517	0.483739839	0.011815952	0.554427114	1.93682E-05	0.307273173	0.526447915	0.149757186
氧化钾(K2O)	0.031225862	0.648713159	1	0.812468583	0.01060598	0.000167163	0.306124256	0.172789953	0.018301227	0.997018419	0.338014579	0.251641102	0.337293816	0.878455406
氧化钙(CaO)	0.000353755	0.009517588	0.812468583	1	0.022675321	0.401552596	0.003327787	0.799962733	0.013405156	0.281604764	0.000238657	0.031150292	0.047999731	0.593256319
氧化镁(MgO)	0.399575589	0.873151433	0.01060598	0.022675321	1	1.13532E-07	0.073966426	0.065153925	0.604379559	0.001786481	0.294891653	0.611019228	0.072174774	0.009329021
氧化铝(Al2O3)	0.002356357	0.319754929	0.000167163	0.401552596	1.13532E-07	1	0.006008182	0.022753838	0.005795693	0.009074408	0.89540787	0.175845842	0.019239255	0.00632413
氧化铁(Fe2O3)	0.764996542	0.158818517	0.306124256	0.003327787	0.073966426	0.006008182	1	0.005660082	0.579357231	0.0014453	0.120446477	0.427097983	0.010281214	0.017971568
氧化铜(CuO)	0.001289055	0.483739839	0.172789953	0.799962733	0.065153925	0.022753838	0.005660082	1	0.522567968	0.000302601	0.127827093	0.210217879	0.012054604	0.001227891
氧化铅(PbO)	3.02276E-10	0.011815952	0.018301227	0.013405156	0.604379559	0.005795693	0.579357231	0.522567968	1	0.495927542	0.010742722	0.010555343	0.529695182	0.460573962
氧化钡(BaO)	0.051714406	0.554427114	0.997018419	0.281604764	0.001786481	0.009074408	0.0014453	0.000302601	0.495927542	1	0.245981201	0.211523784	0.890943384	0.000726891
五氧化二磷(P2O5)	6.74148E-05	1.93682E-05	0.338014579	0.000238657	0.294891653	0.89540787	0.120446477	0.127827093	0.010742722	0.245981201	1	0.077281058	0.822717455	0.16825839
氧化锶(SrO)	3.86712E-05	0.307273173	0.251641102	0.031150292	0.611019228	0.175845842	0.427097983	0.210217879	0.010555343	0.211523784	0.077281058	1	0.924302896	0.117137547
氧化锆(SnO2)	0.501692522	0.526447915	0.337293816	0.047999731	0.072174774	0.019239255	0.010281214	0.012054604	0.529695182	0.890943384	0.822717455	0.924302896	1	0.438605524
二氧化硫(SO2)	0.028332439	0.149757186	0.878455406	0.593256319	0.009329021	0.00632413	0.017971568	0.001227891	0.460573962	0.000726891	0.16825839	0.117137547	0.438605524	1

可得出不呈现显著性的多对变量。其中氧化铁、氧化铜、氧化钡、氧化铅、氧化锶、五氧化二磷、二氧化硫均与多个变量不符合显著性。之后，绘制铅钡 spearman 相关系数热力图：

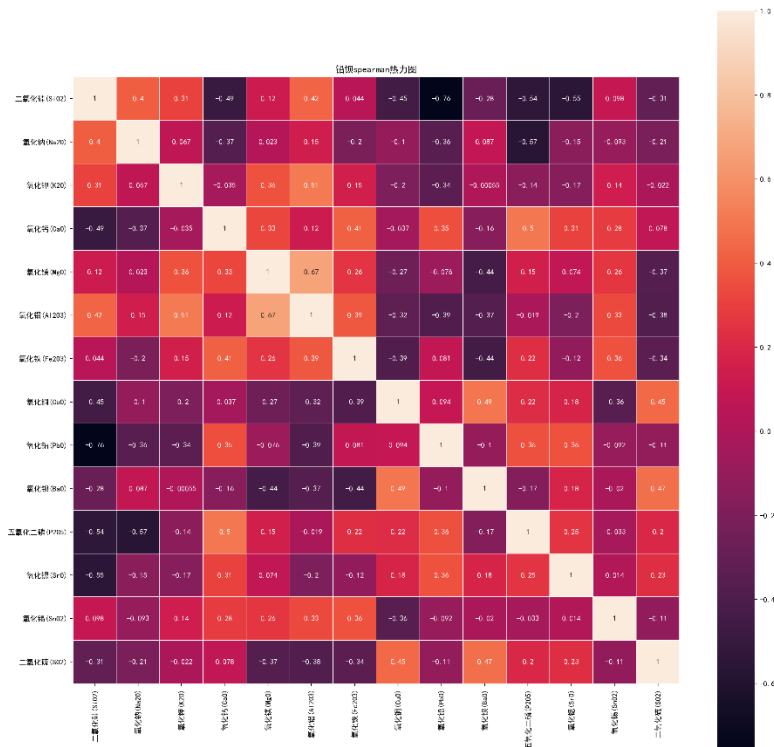


图 17 铅钡 spearman 热力图

表格 17 Spearman 秩相关系数界值表

n	r _{0.05}	r _{0.01}	n	r _{0.05}	r _{0.01}
6	0.886	1.000	31	0.356	0.459
7	0.786	0.929	32	0.350	0.452
8	0.738	0.881	33	0.345	0.446
9	0.700	0.833	34	0.340	0.439
10	0.648	0.794	35	0.335	0.433
11	0.618	0.755	36	0.330	0.427
12	0.587	0.727	37	0.325	0.421
13	0.560	0.703	38	0.321	0.415
14	0.538	0.679	39	0.317	0.410
15	0.521	0.654	40	0.313	0.405
16	0.503	0.635	41	0.309	0.400
17	0.485	0.615	42	0.305	0.395
18	0.472	0.600	43	0.301	0.391
19	0.460	0.584	44	0.298	0.386
20	0.447	0.570	45	0.294	0.382
21	0.435	0.556	46	0.291	0.378
22	0.425	0.544	47	0.288	0.374
23	0.415	0.532	48	0.285	0.370
24	0.406	0.521	49	0.282	0.366
25	0.398	0.511	50	0.279	0.363
26	0.390	0.501			
27	0.382	0.491			
28	0.375	0.483			
29	0.368	0.475			
30	0.362	0.467			

结合 Spearman 秩相关系数界值表（表格 18）， $n=49$ 时 $r \geq 0.282$ 则认为变量之间相关性显著。氧化铝与氧化镁的相关系数高达 0.67，氧化钾与氧化铝的相关系数达到 0.51，五氧化二磷与氧化钙的相关系数达到 0.5，为相关性显著的三对变量。结合显著性检验，可得到该三组的显著性程度高（ $p > 0.05$ ），表明这三组变量的正向关系显著。

最后计算高钾玻璃的 Spearman 秩相关系数。在计算相关系数之前，需要对化学成分之间是否存在统计意义上的显著性关系进行检验，判断 P 值是否有显著性。一般地，判断 P 是否小于 0.05，即在 95% 的置信水平上拒绝原假设，样本有

相关性较为显著。

表格 18 高钾 Spearman 显著性 P 值

	二氧化硅(SiO2)	氧化钠(Na2O)	氧化钾(K2O)	氧化钙(CaO)	氧化镁(MgO)	氧化铝(Al2O3)	氧化铁(Fe2O3)	氧化铜(CuO)	氧化铅(PbO)	氧化钡(BaO)	五氧化二磷(P2O5)	氧化锡(SnO)	氧化锑(SnO2)	二氧化硫(SO2)
二氧化硅(SiO2)	1	0.04639624	0.00013131	0.000321925	0.019192373	1.96126E-05	0.000123328	0.075251678	0.065918048	0.159927054	0.023772421	0.025725245	0.782175109	0.252820598
氧化钠(Na2O)	0.04639624	1	0.007624301	0.004189621	0.373145769	0.206097202	0.385437949	0.6557456	0.28678706	0.346737303	0.378365296	0.664675802	0.669891081	0.430837836
氧化钾(K2O)	0.00013131	0.007624301	1	0.0015256	0.305275622	0.032276322	0.056371321	0.477288438	0.335093797	0.968777238	0.535864314	0.096350008	0.781734441	0.214953507
氧化钙(CaO)	0.000321925	0.004189621	0.0015256	1	0.730535498	0.02647383	0.021659599	0.162166287	0.305150322	0.773419213	0.928674854	0.838705076	0.126082338	0.11023058
氧化镁(MgO)	0.019192373	0.373145769	0.305275622	0.730535498	1	0.000514408	0.013624961	0.416750922	0.234154933	0.026613057	0.003988557	0.002545045	0.393217352	0.07583648
氧化铝(Al2O3)	1.96126E-05	0.206097202	0.032276322	0.02647383	0.000514408	1	0.000362122	0.325644761	0.008184714	0.047287741	0.027546293	0.030084459	0.516510121	0.324689616
氧化铁(Fe2O3)	0.000123328	0.385437949	0.056371321	0.021659599	0.013624961	0.000362122	1	0.004029799	0.191139697	0.021069213	0.028184281	0.091993662	0.126082338	0.205850497
氧化铜(CuO)	0.075251678	0.6557456	0.477288438	0.162166287	0.416750922	0.325644761	0.004029799	1	0.853152397	0.042450521	0.051328638	0.577296534	0.102301278	0.149014771
氧化铅(PbO)	0.065918048	0.28678706	0.335093797	0.305150322	0.234154933	0.008184714	0.191139697	0.853152397	1	0.001833151	0.632559064	0.133294515	0.459675153	0.165484893
氧化钡(BaO)	0.159927054	0.346737303	0.968777238	0.773419213	0.026613057	0.047287741	0.021069213	0.042450521	0.001833151	1	0.048682821	0.014652296	0.611698226	0.346737303
五氧化二磷(P2O5)	0.023772421	0.378365296	0.535864314	0.928674854	0.003988557	0.027546293	0.028184281	0.051328638	0.632559064	0.048682821	1	0.034627441	0.219995101	0.324431808
氧化锡(SnO)	0.025725245	0.664675802	0.096350008	0.838705076	0.002545045	0.030084459	0.091993662	0.577296534	0.133294515	0.014652296	0.034627441	1	0.216456873	0.928746593
氧化锑(SnO2)	0.782175109	0.669891081	0.781734441	0.126082338	0.393217352	0.516510121	0.126082338	0.102301278	0.459675153	0.611698226	0.219995101	0.216456873	1	0.669891081
二氧化硫(SO2)	0.252820598	0.430837836	0.214953507	0.11023058	0.07583648	0.324689616	0.205850497	0.149014771	0.165484893	0.346737303	0.324431808	0.928746593	0.669891081	1

可得不呈现显著性的多对变量。其中，二氧化硫、氧化锡与大部分变量均不呈现显著性，氧化镁、氧化铜、氧化钡、氧化铅、氧化锑、五氧化二磷均与多个变量不符合显著性。

之后，绘制高钾 spearman 相关系数热力图：

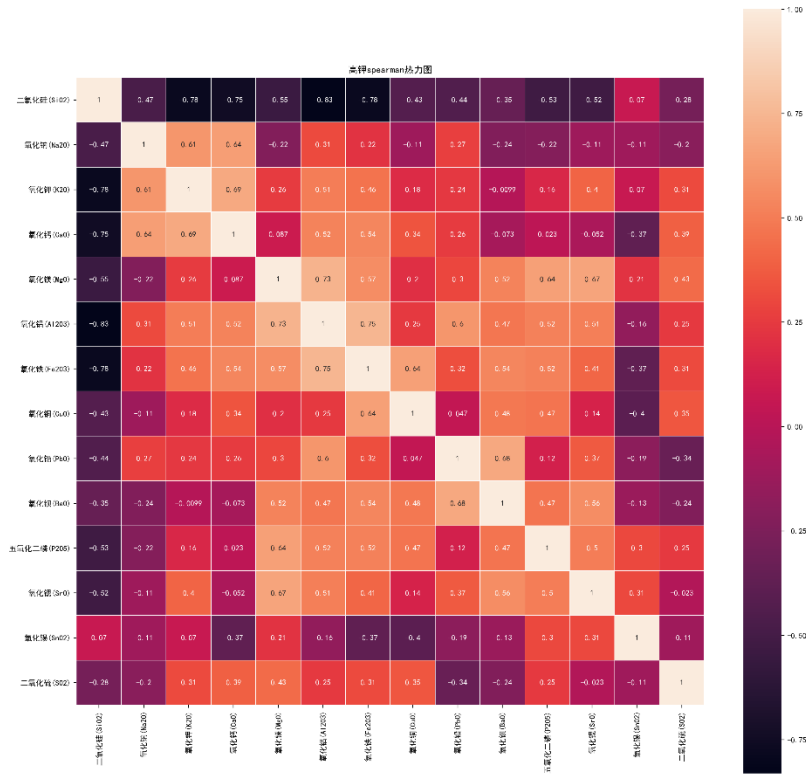


图 18 高钾 spearman 热力图

结合 Spearman 秩相关系数界值表，n=18 时 $r \geq 0.472$ 则认为变量之间相关性显著。氧化铝与氧化铁的相关系数高达 0.75，氧化镁与氧化铝的相关系数高达 0.73，氧化钙与氧化钾的相关系数达到 0.69，多对变量的相关系数超过 0.5。结合显著性检验，可得到该三组的显著性程度高，表明这三组变量的正向关系显著。同时，高钾玻璃的二氧化硅与多个化学成分之间呈现极高的反相关性，相关系数接近-1.00。

5.4.2 化学成分关联关系的差异性

结合铅钡、高钾 Spearman 相关性热力图，可以发现，铅钡玻璃的化学成分之间相关性弱，热力图色调基本为红紫色，意味着大部分化学成分变量的相关系数在 0.00 左右，；而高钾玻璃的化学成分之间相关性强，热力图色调接近白色黑色，意味着大部分化学成分变量的相关系数接近 1.00 或-1.00，正相关或负相关性强。

六、模型评价

6.1 模型的优点与缺点

6.1.1 风化程度与其他属性的关联性模型

缺点：

变量均属于定量变量，文物的纹饰、颜色、类别与风化程度等变量缺乏数值的定量评估，可供选择的数学模型较少，使得该模型的普遍性较弱，推广范围较小。

6.1.2 基于岭回归的化学成分预测模型

优点：

1. 充分考虑实际情况与现实因素，结合化学反应原理构建化学成分预测模型。
2. 模型灵敏度和合理性都较高。

6.1.3 聚类分析模型

优点：

优化改进过后的聚类分析模型基于 LOF 算法，舍弃了不准确的极端数据，在样本数量较少的情况下，保证了较高的准确度。

缺点：

聚类模型中用到的 K-means 算法结果的好坏依赖于初始类中心的选择，常陷入局部最优。

七、模型的改进与推广

1. 关联性检验模型的改进

应增加对变量的定量评估，卡方检验仅提供是否有显著关联性的结果，而

过程不直观。

2. 化学成分预测模型的推广

该推断预测模型同样可用于其他化学组成或者风化过程与玻璃文物相似的文物发掘与保护工作当中。在发掘其他受腐蚀严重的文物时，可以运用此模型。

3. 高维聚类分析模型的改进与推广

该模型同样可以应用于其他特征维数较高且需要将样本分类的情况当中。

八、参考文献

[1] Scientific Platform Serving for Statistics Professional 2021. SPSSPRO. (Version 1.0.11)[Online Application Software]. Retrieved from <https://www.spsspro.com>.

[2] 茆诗松, 王静龙, 濮晓龙, 等. 高等数理统计 (第二版)[M]. 北京:高等教育出版社, 2006.

附录

第一问 堆积直方图

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

sheet1 = pd.read_excel(r"题目/附件.xlsx", sheet_name=0, index_col = 0)
# print(sheet1)

yanse=sheet1["颜色"].value_counts()
x=yanse.index
y=yanse.values
yfh=sheet1[sheet1.表面风化=="风化"].颜色.value_counts()
yy=np.zeros(y.size)

j=0
for i in yanse.index:
    if yfh[yfh.index==i].empty!=True:
        yy[j]=yfh[yfh.index==i].values
```

```

        j=j+1

print(y)
print(yy)

plt.figure(figsize = (8,5))
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.bar(x, y, color = 'lightsteelblue', label="未风化", width=0.7)
plt.bar(x, yy, color = 'royalblue', label="风化", width=0.7)

plt.title("玻璃风化程度与颜色的关系")
plt.xlabel("颜色", fontsize = 14)
plt.ylabel("数量", fontsize = 14)
plt.tick_params(labelsize = 14)
plt.xticks(fontsize=12)

for a,b in zip(x,y):
    plt.text(a,b,b, ha = "center", va = "bottom", fontsize = 10)

plt.legend()
plt.show()

```

第一问 统计规律折线图

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import string

#并表 12 并保存
# sheet1 = pd.read_excel(r"附件 1.xlsx", sheet_name=0)
# sheet2 = pd.read_excel(r"附件 1.xlsx", sheet_name=1)
# hebing=pd.merge(sheet2, sheet1, on=["文物编号"], how="outer")
# resultPath="D:\Personal\Desktop\数学建模\表 12 合并.xlsx"
# hebing.to_excel(resultPath, index = False, na_rep = 0, inf_rep = 0)

hebing=pd.read_excel(r"附件 2.xlsx")
hebing=hebing.drop(["文物采样点", "求和"], axis = 1)
hebing=hebing.fillna(0)

```

```

qianbei=hebing[hebing. 类型=="铅钒"]
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.title("铅钒玻璃化学成分的统计规律")
plt.xlabel("化学成分",fontsize = 14)
plt.ylabel("含量",fontsize = 14)

#均值预测，不需要预测需要注释掉
qianbei1=qianbei[qianbei. 表面风化=="无风化"]
qianbei2=qianbei[qianbei. 表面风化=="风化"]
qianbei1=qianbei1.drop(["类型", "表面风化"],axis = 1)
qianbei2=qianbei2.drop(["类型", "表面风化"],axis = 1)

x=qianbei2.iloc[random.randint(1,qianbei2.shape[0]),:]
print (x)
cha=(qianbei2.mean()-qianbei1.mean())

print("预测值为: ")
print((x-cha).clip(lower=0))

#统计规律折线图
qianbei1=(hebing. 表面风化=="无风化")
wufenghua=qianbei[' 文物采样点'].str.contains(' 未')
wufenghua=qianbei1 | wufenghua
qianbei1=qianbei[wufenghua]
qianbei1=qianbei1.iloc[:-2,:]
qianbei2=qianbei[~wufenghua]

qianbei2=qianbei2.iloc[:,1:-6]
x = np.arange(qianbei2.shape[1]).tolist()
for i in range(qianbei2.shape[0]):
    y=qianbei2.iloc[i].values.tolist()
    if i==0:
        plt.plot(x,y,color='tomato',label="风化")
    else:
        plt.plot(x,y,color='tomato')

qianbei1=qianbei1.iloc[:,1:-6]
x = np.arange(qianbei1.shape[1]).tolist()
for i in range(qianbei1.shape[0]):
    y=qianbei1.iloc[i].values.tolist()
    if i==0:
        plt.plot(x,y,color='lightgreen',label="无风化")

```

```

    else:
        plt.plot(x, y, color='lightgreen')

serious=qianbei['文物采样点'].str.contains('严重')
serious=qianbei[serious]
serious=serious.iloc[:,1:-6]
x = np.arange(serious.shape[1]).tolist()
for i in range(serious.shape[0]):
    y=serious.iloc[i].values.tolist()
    if i==0:
        plt.plot(x, y, color='b', label="严重风化")
    else:
        plt.plot(x, y, color='b')

plt.legend()
plt.show()

```

第二问 聚类分析

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False

sheet = pd.read_excel(r"C:\Users\Lenovo\Desktop\表 12. xlsx", sheet_name=0, index_col = 0)
#print(sheet)

'''
#随机森林分析
#成分
chemical=sheet.iloc[:,0:14]
avg=chemical.mean(axis=0)
std=chemical.std(axis=0)
chemical=(chemical-avg)/std
print(chemical)
#分类
glass=sheet["类型"].values
g=[]
for i in glass:
    if i=="高钾":

```

```

        g.append(1)
    else:
        g.append(0)

#print(glass)
#print(g)

trees=1000      # 森林中几颗树
from sklearn.ensemble import RandomForestClassifier
dtree = RandomForestClassifier(trees,random_state=0)
dtree.fit(chemical, g) #建模
importances = dtree.feature_importances_
#print(importances)

indices = np.argsort(importances)[::-1]

std = np.std([tree.feature_importances_ for tree in dtree.estimators_], axis=0)

plt.bar(range(chemical.shape[1]), importances[indices],
        color="royalblue", align="center",label="重要度")
plt.xticks(range(chemical.shape[1]), chemical.columns[indices],rotation=270)
plt.xlim([-1, chemical.shape[1]])
plt.title("玻璃化学成分重要度")
plt.legend()
plt.show()
#yerr=std[indices]
'''

from sklearn.cluster import KMeans
k=4
kmeans=KMeans(n_clusters=k,max_iter=1000,tol=1e-6)

#高钾 2
gj=sheet[sheet.类型=='高钾'].iloc[:,0:14]
print(gj.iloc[0,5])
avg=gj.mean(axis=0)
std=gj.std(axis=0)
gj=(gj-avg)/std #标准化
from sklearn.neighbors import LocalOutlierFactor as LOF
clf=LOF(n_neighbors=20)
res=clf.fit_predict(gj)
gj=gj[res==1] #去除异常值
print(res)

```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# 返回所有样本的轮廓系数的均值
from sklearn.metrics import silhouette_samples
# 返回的是数据集中每个样本自己的轮廓系数
L = []
for i in range(2, 18):
    k = i
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(gj)
    L.append([i, silhouette_score(gj, kmeans.labels_)])
a = pd.DataFrame(L)
a.columns = ['k', '轮廓系数']
plt.figure(figsize=(8, 5), dpi=70)
plt.plot(a.k, a.轮廓系数, "bo-")
plt.xticks(a.k)
plt.xlabel('k')
plt.ylabel('轮廓系数')
plt.title('高钾-轮廓系数确定最佳的K值', fontdict={'fontsize':15})
plt.show()

from sklearn.cluster import KMeans
k=2
kmeans=KMeans(n_clusters=k, max_iter=1000, tol=1e-6)
kmeans.fit(gj) #建模
label=kmeans.labels_
centroids=kmeans.cluster_centers_

#特征
trees=1000 # 森林中几颗树
from sklearn.ensemble import RandomForestClassifier
dtree = RandomForestClassifier(trees, random_state=0)
dtree.fit(gj, label) #建模
importances = dtree.feature_importances_
indices = np.argsort(importances)[::-1]
plt.bar(range(gj.shape[1]), importances[indices],
        color="royalblue", align="center", label="重要度")
plt.xticks(range(gj.shape[1]), gj.columns[indices], rotation=270)
plt.xlim([-1, gj.shape[1]])
plt.title("高钾玻璃化学成分重要度")
plt.legend()
plt.show()

from PCA import PCA #降维

```

```

pca=PCA(gj)
print(pca.Decompose())
T,P=pca.ConfirmTP(10)

plt.scatter(T[:,0],T[:,1],c=label,s=50,cmap='cool')
#plt.scatter(gj.iloc[:,5],gj.iloc[:,6],c=label,s=50,cmap='cool')
#plt.scatter(centroids[:,5],centroids[:,6],c='red',marker='o',s=75)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('高钾聚类结果')
plt.show()

```

```

#铅钡
qb=sheet[sheet.类型=='铅钡'].iloc[:,0:14]
avg=qb.mean(axis=0)
std=qb.std(axis=0)
qb=(qb-avg)/std #标准化
from sklearn.neighbors import LocalOutlierFactor as LOF
clf=LOF(n_neighbors=20,p=3)
res=clf.fit_predict(qb)
qb=qb[res==1] #去除异常值
print(res)

```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# 返回所有样本的轮廓系数的均值
from sklearn.metrics import silhouette_samples
# 返回的是数据集中每个样本自己的轮廓系数
L = []
for i in range(2, 20):
    k = i
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(qb)
    L.append([i, silhouette_score(qb, kmeans.labels_)])
a = pd.DataFrame(L)
a.columns = ['k', '轮廓系数']
plt.figure(figsize=(8, 5), dpi=70)
plt.plot(a.k, a.轮廓系数, 'bo-')
plt.xticks(a.k)
plt.xlabel('k')
plt.ylabel('轮廓系数')
plt.title('铅钡-轮廓系数确定最佳的K值', fontdict={'fontsize':15})
plt.show()

```

```

from sklearn.cluster import KMeans
k=3
kmeans=KMeans(n_clusters=k,max_iter=1000,tol=1e-6)
kmeans.fit(qb)#建模
label=kmeans.labels_
centroids=kmeans.cluster_centers_

#特征
trees=1000      # 森林中几颗树
from sklearn.ensemble import RandomForestClassifier
dtree = RandomForestClassifier(trees,random_state=0)
dtree.fit(qb, label) #建模
importances = dtree.feature_importances_
indices = np.argsort(importances)[::-1]
plt.bar(range(qb.shape[1]), importances[indices],
        color="royalblue", align="center",label="重要度")
plt.xticks(range(qb.shape[1]), qb.columns[indices],rotation=270)
plt.xlim([-1, qb.shape[1]])
plt.title("铅钡玻璃化学成分重要度")
plt.legend()
plt.show()

from PCA import PCA #降维
pca=PCA(qb)
print(pca.Decompose())
T,P=pca.ConfirmTP(6)

plt.scatter(T[:,0],T[:,1],c=label,s=50,cmap='cool')
#plt.scatter(qb.iloc[:,8],qb.iloc[:,0],c=label,s=50,cmap='cool')
#plt.scatter(centroids[:,8],centroids[:,0],c='red',marker='o',s=75)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('铅钡聚类结果')
plt.show()

```

第三问 PCR

```
import numpy as np
import pandas as pd
from sklearn.metrics import f1_score, precision_score, recall_score

#读取数据
sheet = pd.read_excel(r"C:\Users\Lenovo\Desktop\表 12. xlsx", sheet_name=0, index_col = 0)
sheet1=sheet[sheet.表面风化=="无风化"]
x1=sheet1.iloc[:,0:14]
y1=sheet1["类型"]
,,,

y1=[]
count1=sheet1["类型"].values
print(type(y1))
for i in count1:
    if i=="高钾":
        y1.append([1,0])
    else:
        y1.append([0,1])
,,,

sheet2=sheet[sheet.表面风化=="风化"]
x2=sheet2.iloc[:,0:14]
y2=sheet2["类型"]
test=pd.read_excel(r"C:\Users\Lenovo\Desktop\附件 3. xlsx", sheet_name=2, index_col = 0)
test1=test[test.表面风化=="无风化"].iloc[:,1:15].fillna(0)
test2=test[test.表面风化=="风化"].iloc[:,1:15].fillna(0)

#交叉验证
from sklearn.model_selection import KFold
folds=10
kf = KFold(n_splits=folds) # 10 份交叉验证
#无风化
yTrue=None; yHat=None
for trainIndex, testIndex in kf.split(x1):
    Xtrain, Xtest = x1.iloc[trainIndex], x1.iloc[testIndex]
    ytrain, ytest = y1.iloc[trainIndex], y1.iloc[testIndex]
    #数据预处理
    aver=Xtrain.mean(axis=0) # 取每列的平均值
    std=Xtrain.std(axis=0) # 每列的标准差
    Xtrain=(Xtrain-aver)/std
    ynew=[[1,0] if item=="高钾" else [0,1] for item in ytrain]
```

```

ytest=[0 if item=="高钾" else 1 for item in ytest]
#pcr 建模
from PCR import PCR
pcr=PCR(Xtrain,ynew)
pcr.confirmPCs()
pcr.fit(12)
#预测
Xtest=(Xtest-aver)/std
yPred=pcr.predict(Xtest)
exp=np.exp(yPred)
sumExp=exp.sum(axis=1,keepdims=True)
softmax=exp/sumExp
yPredClass=np.argmax(softmax,axis=1)
#预测
test1=(test1-aver)/std
result1=pcr.predict(test1)
exp=np.exp(result1)
sumExp=exp.sum(axis=1,keepdims=True)
softmax=exp/sumExp
resultPredClass=np.argmax(softmax,axis=1)
print(softmax)
print(resultPredClass)
if yTrue is None: # 第一份预测, 当时 yTrue 是 None
    yTrue= ytest # 真值
    yHat=yPredClass #预测值
else:
    yTrue=np.r_[yTrue,ytest] # 后续预测, 进行行叠加
    yHat=np.r_[yHat, yPredClass]
f1 = f1_score(yTrue, yHat, average='macro')
sen=len(yHat[yHat==0])/len(yTrue[yTrue==0])
err=yTrue-yHat
err=err[err!=0]
errRate=len(err)/len(y1)
print(errRate)
print(f1)
print(sen)

folds=10
kf = KFold(n_splits=folds) # 10 份交叉验证
#风化
y2True=None; y2Hat=None
for trainIndex, testIndex in kf.split(x2):
    Xtrain, Xtest = x2.iloc[trainIndex], x2.iloc[testIndex]
    ytrain, ytest = y2.iloc[trainIndex], y2.iloc[testIndex]

```

```

#数据预处理
aver=Xtrain.mean(axis=0)    # 取每列的平均值
std=Xtrain.std(axis=0)      # 每列的标准差
Xtrain=(Xtrain-aver)/std
ynew=[[1,0]if item=="高钾" else[0,1]for item in ytrain]
ytest=[0 if item=="高钾" else 1 for item in ytest]
#pcr 建模
from PCR import PCR
pcr=PCR(Xtrain,ynew)
pcr.confirmPCs()
pcr.fit(4)
#预测
Xtest=(Xtest-aver)/std
yPred=pcr.predict(Xtest)
exp=np.exp(yPred)
sumExp=exp.sum(axis=1,keepdims=True)
softmax=exp/sumExp
yPredClass=np.argmax(softmax,axis=1)
#预测
test2=(test2-aver)/std
result2=pcr.predict(test2)
exp=np.exp(result2)
sumExp=exp.sum(axis=1,keepdims=True)
softmax=exp/sumExp
resultPredClass=np.argmax(softmax,axis=1)
print(softmax)
print(resultPredClass)
if y2True is None: # 第一份预测, 当时 yTrue 是 None
    y2True= ytest # 真值
    y2Hat=yPredClass #预测值
else:
    y2True=np.r_[y2True,ytest] # 后续预测, 进行行叠加
    y2Hat=np.r_[y2Hat, yPredClass]
f1 = f1_score(y2True, y2Hat, average='macro')
sen2=len(y2Hat[y2Hat==1])/len(y2True[y2True==1])
err=y2True-y2Hat
err=err[err!=0]
errRate=len(err)/len(y2)
print(errRate)
print(f1)
print(sen2)

```

第三问 PSL

```

import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import KFold
from sklearn.metrics import f1_score, precision_score, recall_score

#读取数据
sheet = pd.read_excel(r"C:\Users\Lenovo\Desktop\表 12. xlsx", sheet_name=0, index_col = 0)
sheet1=sheet[sheet.表面风化=="无风化"]
x1=sheet1.iloc[:,0:14]
y1=sheet1["类型"]
sheet2=sheet[sheet.表面风化=="风化"]
x2=sheet2.iloc[:,0:14]
y2=sheet2["类型"]
test=pd.read_excel(r"C:\Users\Lenovo\Desktop\附件 3. xlsx", sheet_name=2, index_col = 0)
test1=test[test.表面风化=="无风化"].iloc[:,1:15].fillna(0)
test2=test[test.表面风化=="风化"].iloc[:,1:15].fillna(0)

def cvPCS(n): # n 是主成分数
    yTrue=None; yHat=None
    #pls 建模
    pls = PLSRegression(n_components=n, scale=False)
    for trainIndex, testIndex in kf.split(x1):
        Xtrain, Xtest = x1.iloc[trainIndex], x1.iloc[testIndex]
        ytrain, ytest = y1.iloc[trainIndex], y1.iloc[testIndex]
        #数据预处理
        aver=Xtrain.mean(axis=0) # 取每列的平均值
        std=Xtrain.std(axis=0) # 每列的标准差
        Xtrain=(Xtrain-aver)/std
        ynew=[[1,0]if item=="高钾" else [0,1]for item in ytrain]
        ytest=[0 if item=="高钾" else 1 for item in ytest]
        #拟合建模
        pls.fit(Xtrain, ynew)
        #预测
        Xtest=(Xtest-aver)/std
        yPred=pls.predict(Xtest)
        exp=np.exp(yPred)
        sumExp=exp.sum(axis=1,keepdims=True)
        softmax=exp/sumExp
        yPredClass=np.argmax(softmax,axis=1)
        #预测
        global test1
        test1=(test1-aver)/std
        result1=pls.predict(test1)

```

```

exp=np.exp(result1)
sumExp=exp.sum(axis=1,keepdims=True)
softmax=exp/sumExp
resultPredClass=np.argmax(softmax,axis=1)
print(softmax)
print(resultPredClass)
if yTrue is None: # 第一份预测, 当时 yTrue 是 None
    yTrue= ytest # 真值
    yHat=yPredClass # 预测值
else:
    yTrue=np.r_[yTrue,ytest] # 后续预测, 进行叠加
    yHat=np.r_[yHat, yPredClass]
    f1 = f1_score(yTrue, yHat, average='macro')
err=yTrue-yHat
err=err[err!=0]
errRate=len(err)/len(y1)
sen=sen=len(yHat[yHat==0])/len(yTrue[yTrue==0])
return sen

def cvPCS2(n): # n 是主成分数
    yTrue=None; yHat=None
    #pls 建模
    pls = PLSRegression(n_components=n, scale=False)
    for trainIndex, testIndex in kf.split(x2):
        Xtrain, Xtest = x2.iloc[trainIndex], x2.iloc[testIndex]
        ytrain, ytest = y2.iloc[trainIndex], y2.iloc[testIndex]
        #数据预处理
        aver=Xtrain.mean(axis=0) # 取每列的平均值
        std=Xtrain.std(axis=0) # 每列的标准差
        std[std==0]=1
        Xtrain=(Xtrain-aver)/std
        ynew=[[1,0] if item=="高钾" else [0,1] for item in ytrain]
        ytest=[0 if item=="高钾" else 1 for item in ytest]
        #拟合建模
        pls.fit(Xtrain, ynew)
        #预测
        Xtest=(Xtest-aver)/std
        yPred=pls.predict(Xtest)
        exp=np.exp(yPred)
        sumExp=exp.sum(axis=1,keepdims=True)
        softmax=exp/sumExp
        yPredClass=np.argmax(softmax,axis=1)
        #预测
        global test2

```

```

test2=(test2-aver)/std
result2=pls.predict(test2)
exp=np.exp(result2)
sumExp=exp.sum(axis=1, keepdims=True)
softmax=exp/sumExp
resultPredClass=np.argmax(softmax, axis=1)
print(softmax)
print(resultPredClass)
if yTrue is None: # 第一份预测, 当时 yTrue 是 None
    yTrue= ytest # 真值
    yHat=yPredClass # 预测值
else:
    yTrue=np.r_[yTrue, ytest] # 后续预测, 进行叠加
    yHat=np.r_[yHat, yPredClass]
f1 = f1_score(yTrue, yHat, average='macro')
err=yTrue-yHat
err=err[err!=0]
errRate=len(err)/len(y2)
sen=sen=len(yHat[yHat==0])/len(yTrue[yTrue==0])
print(errRate)
return f1
print(len(y1))
print(len(y2))
#交叉验证
folds=10
kf = KFold(n_splits=folds) # 10 份交叉验证
pcs=list(range(1, 14))
,,,
erro=[]
for i in pcs:
    erro.append(cvPCS(i))
,,,
print(cvPCS2(3))
,,,

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.plot(pcs, erro)
plt.xlabel('主成分')
plt.ylabel('误差率')
plt.title('主成分个数与误差关系', fontdict={'fontsize':15})
plt.show()
,,,

```

第四问 spearman 热力图

```
import seaborn as sns
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import string

data=pd.read_excel(r"附件 3.xlsx")
data=data.fillna(0)

qianbei=data[data.类型=="铅钒"]
qianbei.drop(["类型"],axis = 1,inplace=True)

#正态性检验
s=[]
p=[]
from scipy.stats import shapiro
for i in range(qianbei.shape[1]):
    ss,pp=shapiro(qianbei.iloc[:,i])
    s.append(ss)
    p.append(pp)
    if(pp<0.05):
        print(qianbei.columns.values[i],"不符合正态分布")

plt.figure(figsize=(18,18))
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
sns.heatmap(qianbei.corr(method='spearman'),linewidths=0.1,vmax=1.0,
square=True,linecolor='white', annot=True)
plt.title('高钾 spearman 热力图')
plt.show()

def GetValue(x,y):
    return spearmanr(x,y)[1]

from scipy.stats import spearmanr
ans=qianbei.corr(method=GetValue)
for i in range(14):
    for j in range(14):
        if ans.iloc[i,j]==1:
            break
        elif ans.iloc[i,j]>=0.05:
```

```
print(ans.index.values[i], ans.columns.values[j], ans.iloc[i, j])
```